

UNIT-4. DATA STRUCTURES

4.1 Explain Python Lists:

A sequence is a datatype that represents a group of elements. The purpose of any sequence is to store and process group elements. In python, strings, lists, tuples and dictionaries are very important sequence datatypes.

LIST:

A List is data type which can store different types of elements. And we can also store duplicate data also.

Creating a List:

Creating a list is as simple as putting different comma-separated values between square brackets [].

```
student = [556, 'Moithi', 84, 96, 84, 75, 84]
```

We can create empty list without any elements by simply writing empty square brackets as: `student=[]`

We can create a list by embedding the elements inside a pair of square braces []. The elements in the list should be separated by a comma (,).

Accessing/Reading Values in list:

To access values in lists, use the square brackets for slicing along with the index or indices to obtain value available at that index. To view the elements of a list as a whole, we can simply pass the list name to print function.

negative Indexing	-7	-6	-5	-4	-3	-2	-1
Positive Indexing	0	1	2	3	4	5	6
Student	556	"Moithi"	84	96	84	75	84

Ex:

```
student = [556, 'Moithi', 84, 96, 84, 75, 84]
print(student)
print(student[0]) # Access 0th element
print(student[0:2]) # Access 0th to 1st elements
print(student[2: ]) # Access 2nd to end of list elements
print(student[ :3]) # Access starting to 2nd elements
print(student[ : ]) # Access starting to ending elements
print(student[-1]) # Access last index value
print(student[-1:-7:-1]) # Access elements in reverse order
```

Output:

```
[556, 'Moithi', 84, 96, 84, 75, 84]
Moithi
```

```
[556, ,Moithi']
[84, 96, 84, 75, 84]
[556, ,Moithi', 84]
[556, ,Moithi', 84, 96, 84, 75, 84]
84
[84, 75, 84, 96, 84, ,Moithi']
```

Creating lists using range() function:

We can use range() function to generate a sequence of integers which can be stored in a list. To store numbers from 0 to 10 in a list as follows.

```
numbers = list( range(0,11) )
print(numbers)          # [0,1,2,3,4,5,6,7,8,9,10]
```

To store even numbers from 0 to 10 in a list as follows.

```
numbers = list( range(0,11,2) )
print(numbers) # [0,2,4,6,8,10]
```

List Basic operations:

Updating and deleting lists:

Lists are *mutable*. It means we can modify the contents of a list. We can append, update or delete the elements of a list depending upon our requirements.

Appending an element means adding an element at the end of the list. To, append a new element to the list, we should use the append() method.

Example:

```
lst=[4,7,6,8,9,3]
print(lst)          # [4,7,6,8,9,3]
lst[2]=5            # updates 2nd element in the list
print(lst)          # [4,7,5,8,9,3]
lst[2:5]=10,11,12   # update 2nd element to 4th element in the list
print(lst)          # [4,7,10,11,12,3]
```

Deleting an element from the list can be done using 'del' statement. The del statement takes the position number of the element to be deleted.

Example:

```
lst=[5,7,1,8,9,6]
del lst[3] # delete 3rd element from the list i.e., 8
print(lst) # [5,7,1,9,6]
```

If we want to delete entire list, we can give statement like **del lst**.

Concatenation of Two lists:

We can simply use „+“ operator on two lists to join them. For example, „x“ and „y“ are two lists. If we write x+y, the list „y“ is joined at the end of the list „x“.

Example:

```
x=[10,20,32,15,16]
y=[45,18,78,14,86]
print(x+y)          # [10,20,32,15,16,45,18,78,14,86]
```

Repetition of Lists:

We can repeat the elements of a list „n“ number of times using „*“ operator.

```
x=[10,54,87,96,45]
print(x*2) # [10,54,87,96,45,10,54,87,96,45]
```

Membership in Lists:

We can check if an element is a member of a list by using „in“ and „not in“ operator. If the element is a member of the list, then „in“ operator returns **True** otherwise returns **False**. If the element is not in the list, then „not in“ operator returns **True** otherwise returns **False**.

Example:

```
x=[10,20,30,45,55,65]
a=20
print(a in x)      # True
a=25
print(a in x)      #
False a=45
print(a not in x) #
False a=40
print(a not in x) # True
```

Explain List slices:

The format for list slicing is [start:stop:step]start is the index of the list where slicing starts. Stop is the index of the list where slicing ends. step allows you to select nth item within the range start to stop. List slicing works similar to string slice() in python.

Original List:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Sliced Lists:

```
[4, 6, 8]
```

```
[1, 3, 5, 7, 9]
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

EXAMPLE1:

OUTPUT:

# Initialize list	Original list :
List = [1,2,3,4,5,6,7,8,9]	[1,2,3,4,5,6,7,8,9]
# Show original list	Sliced
Lists: print("\nOriginal List:\n", List)	[4,
6, 8]	
print("\nSliced Lists: ")	[1, 3, 5, 7, 9]
# Display sliced list	[1,2,3,4,5,6,7,8,9]
print(List[3:9:2])	
# Display sliced list	
print(List[:2])	
# Display sliced list	
print(List[::])	

EXAMPLE2:

```

List = ['ccn','branch','python']
# Show original list
print("\nOriginal List:\n", List)
print("\nSliced Lists: ")
# Display sliced list
print(List[::-1])
# Display sliced list
['python'] print(List[::-3])
# Display sliced
list
print(List[1:-2])

```

OUTPUT:

```

Original List:
['ccn','branch','python']
Sliced Lists:
['python','branch','ccn']
['python']

```

Explain list methods:

- **A.index(X)**: Returns the first occurrence of X in the list A.

Program	Output
A=[1,3,13,45,56,62,45,58,89,13,3,56,1] print(A.index(45))	3

- **A.append(X)**: Adds the element X at the end of list A.

Program	Output
A=[1,3,13,45,56,62,45,58,89,13,3,56,1] A.append(97) print(A)	[1, 3, 13, 45, 56, 62, 45, 58, 89, 13, 3, 56, 1, 97]

- **A.insert(i,X)**: Inserts X to the list A in ith position.

Program	Output
A=[1,3,13,45,56,62,45,58,89,13,3,56,1] A.insert(4,"CSE") print(A)	[1, 3, 13, 45, 'CSE', 56, 62, 45, 58, 89, 13, 3, 56, 1]

- **B=A.copy()**: copies all the list elements of A into B.

Program	Output
A=[1,3,13,45,56,62,45,58,89,13,3,56,1] B=A.copy() print("B=", B)	B= [1, 3, 13, 45, 56, 62, 45, 58, 89, 13, 3, 56, 1]

- **A.extend(B)**: Appends list B to list A.

Program	Output
A=[1,3,13,45,56,] B=[62,45,58,89,13,3,56,1] A.extend(B) print("A=",A) print("B=",B)	A= [1, 3, 13, 45, 56, 62, 45, 58, 89, 13, 3, 56, 1] B= [62, 45, 58, 89, 13, 3, 56, 1]

- **A.count(X)**: Returns number of occurrences of X in the list A.

Program	Output
A=[1,3,13,45,56,62,45,58,89,13,3,56,1] print(A.count(45))	2

- **A.remove(X)**: Removes X from the list X.

Program	Output
A=[1,3,13,45,56,62,45,58,89,13,3,56,1] A.remove(13) print(A)	[1, 3, 45, 56, 62, 45, 58, 89, 13, 3, 56, 1]

- **A.pop()**: Removes the ending element from list A.

Program	Output
A=[1,3,13,45,56,62,45,58,89,13,3,56,1] A.pop() print(A)	[1, 3, 13, 45, 56, 62, 45, 58, 89, 13, 3, 56]

- **A.sort()**: Sorts the elements of list A into ascending order.

Program	Output
A=[1,3,13,45,56,62,45,58,89,13,3,56,1] A.sort() print(A)	[1, 1, 3, 3, 13, 13, 45, 45, 56, 56, 58, 62, 89]

- **A.reverse()**: Reverses the sequence of elements in the list.

Program	Output
A=[1,3,13,45,56,62,45,58,89,13,3,56,1] A.reverse() print(A)	[1, 56, 3, 13, 89, 58, 45, 62, 56, 45, 13, 3, 1]

- **A.clear()**: Removes all elements from the list A.

Program	Output
A=[1,3,13,45,56,62,45,58,89,13,3,56,1] A.clear() print(A)	[]

- **max(A)**: Returns biggest element in the list A.

Program	Output
A=[1,3,13,45,56,62,45,58,89,13,3,56,1] print(max(A))	89

- **min(A)**: Returns smallest element in the list B.

Program	Output
A=[1,3,13,45,56,62,45,58,89,13,3,56,1] print(min(A))	1

- **len(A)**: Returns the length of list A

Program	Output
A=[1,2,3,4,5] print(len(A))	5

4.5 Looping on lists:

We can also display list by using for loop (or) while loop. The len() function useful to know the numbers of elements in the list. while loop retrieves starting from 0th to the last element i.e. n-1

Ex-1:

```
numbers = [1,2,3,4,5]
for i in numbers:
    print(i,end="")
```

Output:

12345

Ex-2:

```
list = [1, 3, 5, 7, 9]
# Getting length of list
length = len(list)
i = 0

while i < length:
    print(list[i])
    i += 1
```

Output:

1
3
5
7
9

Explain mutability:

A mutable object can be changed but an immutable object cannot. As you know, an integer is an immutable type in Python. Let me demonstrate how you can realize this by running some experiments. First, let's create two variables a and b. The variable a is an integer, and the variable b refers to a:

```
a = 1
b = a
```

Now both a and b point to the same memory address. In other words, the id() function should return the same ID for both integers:

```
a = 1
b = a
print(id(a) == id(b))
```

Output:

True

And this is indeed the case.

But now, let's change the value of a:

```
a = 10
```

Now, let's compare the IDs of a and b again:

```
print(id(a) == id(b))
```

Output:

False

The IDs don't match anymore. This is because now `a` points to a different integer object. In other words, the integer object 1 itself never changed. But the variable `a` that pointed to it now points to another integer object called 10.

So even though it looks as if you updated the original integer object, you did not. This is due to the fact that an integer is an immutable type in Python, and you cannot change an immutable object after creation.

Now, let's repeat a similar experiment with a mutable object.

For example, let's create a list:

```
nums = [1, 2, 3]
```

```
l = nums
```

Now, let's compare the IDs of the lists:

```
print(id(nums) == id(l))
```

Output:

```
True
```

Now, let's modify the list object by removing its first element:

```
del nums[0]
```

Let's then check if the IDs of the two lists still match:

```
print(id(nums) == id(l))
```

Result:

```
True
```

And they do! This means `nums` and `l` still point to the same list object in memory.

To make the point clear, let's print the contents of `nums` and `l`:

```
print(nums  
) print(l)
```

Output:

```
[2, 3]
```

```
[2, 3]
```

&4.8.Explain aliasing and cloning:

Aliasing and Cloning Lists:

Giving a new name to an existing list is called 'aliasing'. The new name is called 'alias name'. To provide a new name to this list, we can simply use assignment operator (`=`).

Example:

```
x = [10, 20, 30, 40, 50, 60]
```

```
y=x # x is aliased as y
```

```
print x #
```

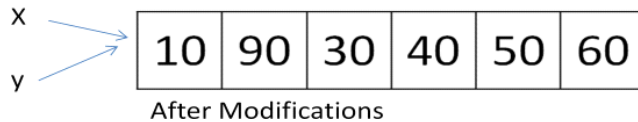
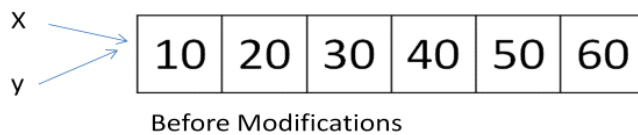
```
[10,20,30,40,50,60] print y
```

```
# [10,20,30,40,50,60]
```

```
x[1]=90 # modify 1st element in
```

```
x print x # [10,90,30,40,50,60]
```

```
print y # [10,90,30,40,50,60]
```

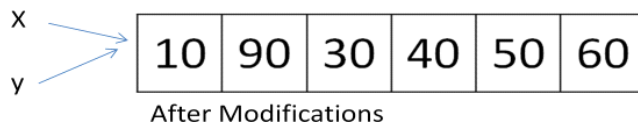
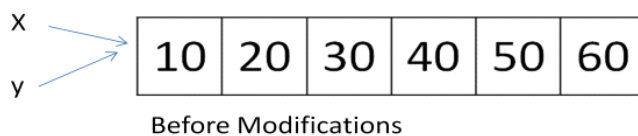


In this case we are having only one list of elements but with two different names „x“ and „y“. Here, „x“ is the original name and „y“ is the alias name for the same list. Hence, any modifications done to x“ will also modify „y“ and vice versa.

Obtaining exact copy of an existing object (or list) is called „cloning“. To Clone a list, we can take help of the slicing operation [:].

Example:

```
x = [10, 20, 30, 40, 50, 60]
y=x[:] # x is cloned as y
print x # [10,20,30,40,50,60]
print y # [10,20,30,40,50,60]
x[1]=90 # modify 1st element in x
print x # [10,90,30,40,50,60]
```



```
print y # [10,20,30,40,50,0]
```

When we clone a list like this, a separate copy of all the elements is stored into „y“.

The lists „x“ and „y“ are independent lists. Hence, any modifications to „x“ will not affect „y“ and vice versa.

Explain list parameters:

In python we can pass list as a parameters. in arguments what type of data is passed to function, inside function also treated as same type.

syntax: list(iterable)

here iterable means sequence of elements.

here if we cannot pass any values of list function , then it doesn't shown an error . we can pass list values to a function

Example 1:To pass a list to function.

```
>>>def color(b):
>>>    for i in b:
>>>        print(i)
>>>cr1=["red","blue","green"]
```

OUTPUT:

```
red
blue
green
```

```
>>>color(cr1)
```

Example 2:**OUTPUT 2:**

```
>>>def value(b):  
>>>    print(b)  
>>>l=[10,20,3+5j,"mouni"]  
>>>value(l)
```

```
[10,20,3+5j,"mouni"]
```

the above example for to pass different data values to function.

Example 3:**OUTPUT3:**

```
>>>def sort(a,b):  
>>>    c=a+b  
>>>    c.sort()  
>>>    return c  
>>>a1=[10,5,25,30,4]  
>>>a2=[6,7,40,35,71]  
>>>res=sorting(a1,a2)  
>>>print(res)
```

```
[4,5,6,7,10,25,30,35,40,71]
```

the above example for to pass two different lists to function.that are a1 and a2

Example 4:**OUTPUT4:**

```
>>>def count(b):  
>>>    even=0  
>>>    odd=0  
>>>    for i in b:  
>>>        if i%2==0:  
>>>            even+=1  
>>>        else:  
>>>            odd+=1  
>>>    return even,odd  
>>>a=[5,10,7,8,6,9,13]  
>>>even,odd=count(a)  
>>>print("even is {} and odd is {}".format(even,odd))
```

```
even is 3 and odd is 4
```

The above example to pass a list to function and it returns to more than one value.

Example 5:**OUTPUT5:**

```
>>>def expression(n):  
>>>    a=sum(n)  
>>>    return a  
>>>x=[10,20,30,40,50]  
>>>r=expression(x)  
>>>print("the sum of list is ",r)
```

```
the sum of list is 150
```

Explain list comprehension:

List comprehensions represent creation of new lists from an iterable object (like a list, set, tuple, dictionary or range) that satisfy a given condition. List comprehensions contain very compact code usually a single statement that performs the task. A Python list comprehension consists of brackets containing the expression, which is executed for each element along with the for loop to iterate over each element in the Python list.

Python List comprehension provides a much more short syntax for creating a new list based on the values of an existing list.

Advantages of List Comprehension

- More time-efficient and space-efficient than loops.
- Require fewer lines of code.
- Transforms iterative statement into a formula.

Syntax of List Comprehension

`newList = [ expression(element) for element in oldList if condition ]`

We want to create a list with squares of integers from 1 to 100. We can write code as:

```
squares=[ ]  
for i in range(1,11):  
    squares.append(i**2)
```

The preceding code will create „squares“ list with the elements as shown below:

```
[ 1, 4, 9, 16, 25, 36, 49, 64, 81, 100 ]
```

The previous code can be rewritten in a compact way as:

```
squares=[x**2 for x in range(1,11)]
```

This is called list comprehension. From this, we can understand that a list comprehension consists of square braces containing an expression (i.e., `x**2`). After the expression, a for loop and then zero or more if statements can be written.

Example:

```
Even_squares = [ x**2 for x in range(1,11) if x%2==0 ]
```

It will display the list even squares as list.

```
[ 4, 16, 36, 64, 100 ]
```

Example 3: Matrix using List comprehension

```
matrix = [[j for j in range(3)] for i in range(3)]  
  
print(matrix)
```

Output:
[[0, 1, 2], [0, 1, 2], [0, 1, 2]]

TUPLES:

A Tuple is a python sequence which stores a group of elements or items. Tuples are similar to lists but the main difference is **tuples are immutable** whereas lists are mutable. Once we create a tuple we **cannot modify** its elements. Hence, we **cannot perform** operations like `append()`, `extend()`, `insert()`, `remove()`, `pop()` and `clear()` on tuples. Tuples are generally used to store data which should not be modified and retrieve that data on demand.

Creating Tuples:

We can create a tuple by writing elements separated by commas inside parentheses().

The elements can be same datatype or different types.

```
tup=(10, 20, 31.5, 'aditya')
```

If we do not mention any brackets and write the elements separating them by comma, then they are taken by default as a **tuple**. `tup= 10, 20, 34, 47`

It is possible to create a tuple from a list. This is done by converting a list into a tuple

using tuple function.

```
t=tuple(range(2,11,2))
print(t)    # display (2,4,6,8,10)
```

Another way to create a tuple by using range() function that returns a sequence.

Tuple Assignment:

In python, we can perform tuple assignment which is a quite useful feature. We can create a tuple in various ways We also call this feature **unpacking** of tuple.

The process of assigning values to a tuple is known as **packing**. In unpacking, we basically extract the values of the tuple into a single variable.

Tuple Packing (Creating Tuples)

We can create a tuple in various ways by using different types of elements. Since a tuple can contain all elements of the same data type as well as of mixed data types as well. Let us look at few examples of creating tuples in python which we consider as packing.

Example 1: Tuple with integers as elements

```
>>>tup = (22, 33, 5, 23)
>>>tup
(22, 33, 5, 23)
```

Example 2: Tuple with mixed data type

```
>>>tup2 = ('hi', 11, 45.7)
>>>tup2
('hi', 11, 45.7)
```

Example 3: Tuple with a tuple as an element

```
>>>tup3 = (55, (6, 'hi'), 67)
>>>tup3
(55, (6, 'hi'), 67)
```

Example 4: Tuple with a list as an element

```
>>>tup3 = (55, [6, 9], 67)
>>>tup3
(55, [6, 9], 67)
```

If there is only a single element in a tuple we should end it with a comma. Since writing, just the element inside the parenthesis will be considered as an integer.

For example,

```
>>>tup=(90)
>>>tup
90
>>>type(tup)
<class 'int'>
```

Correct way of defining a tuple with single element is as follows:

```
>>>tup=(90,)
>>>tup
(90,)
>>>type(tup)
<class 'tuple'>
```

Moreover, if you write any sequence separated by commas, python considers it as a tuple.

For example,

```
>>> seq = 22, 4, 56
```

```
>>>seq
(22, 4, 56)
>>>type(seq)
<class 'tuple'>
```

Tuple Assignment (Unpacking)

Unpacking or tuple assignment is the process that assigns the values on the right-hand side to the left-hand side variables. In unpacking, we basically extract the values of the tuple into a single variable.

NOTE : In unpacking of tuple number of variables on left-hand side should be equal to number of values in given tuple a.

Example 1

```
>>>(n1, n2) = (99, 7)
>>>print(n1)
99
>>>print(n2)
7
```

Example 2

```
>>>tup1 = (8, 99, 90, 6.7)
>>>(roll no., english, maths, GPA) = tup1
>>>print(english)
99
>>>print(roll no.)
8
>>>print(GPA)
6.7
>>>print(maths)
90
```

Example 3

```
>>> (num1, num2, num3, num4, num5) = (88, 9.8, 6.8, 1)
#this gives an error as the variables on the left are more than the number of elements in
the tuple
Value Error: not enough values to unpack (expected 5, got 4)
```

Using Astersik(*)

```
fruits = ("apple", "banana", "cherry", "strawberry", "raspberry")
(green, yellow, *red) = fruits
print(green)
print(yellow)
print(red)
```

output: apple

banana

["cherry", "strawberry", "raspberry"]

Tuple as return value

A function can only return one value, but if the value is a tuple, the effect is the same as returning multiple values. For example, if you want to divide two integers and compute the quotient and remainder, it is inefficient to compute x/y and then $x\%y$. It is better to compute them both at the

same time.

The built-in function `divmod` takes two arguments and returns a tuple of two values, the quotient and remainder. You can store the result as a tuple:

```
>>> t = divmod(7, 3)
>>> print(t)
>>> (2, 1)
```

Or use tuple assignment to store the elements separately:

```
>>> quot, rem = divmod(7, 3)
>>> print(quot)
>>> 2
>>> print(rem)
>>> 1
```

By using user defined functions

EXAMPLE:

```
def calc():
    a=int(input("enter a value"))
    b=int(input("enter b value"))
    c=a+b
    d=a-b
    e=a*b
    return c,d,e
a1,a2,a3=calc()
print(a1)
print(a1)
print(a1)
```

OUTPUT:

```
enter a value 10
enter b value 5
15
5
50
```

Tuple comprehension

Comprehension is a technique used to create a new sequence from already existing sequence with short and concise way. there is no tuple comprehension in python because comprehension requires mutable data type. in python there is no syntax for tuple comprehension but indirectly we can perform tuple comprehension. it is done by using the following types

1. wrapping generator comprehension
2. list comprehension

Example 1:

```
>>>t1=(5,10,15,24,26)
>>>result=(x for x in t1 if x%5==0)
>>>print(result)
>>> <generator object <genexpr> at 0x000001D48DEC6F80>
```

Example 2:

```
>>>t1=(5,10,15,24,26)
>>>result=tuple(x for x in t1 if x%5==0)
>>>print(result)
>>>(5,10,15)
```

Example 3:

```
>>>t1=[1,2,3,4,5]
>>>r=tuple([value*10 for value in t1])
>>>print(r)
>>>(10,20,30,40,50)
```

Dictionary:

A dictionary represents a group of elements arranged in the form of key-value pairs.

The first element is considered as „key“ and the immediate next element is taken as its „value“. The key and its value are separated by a colon (:). All the key-value pairs in a dictionary are inserted in curly braces { }.

- Keys should be unique. It means, duplicate keys are not allowed. If we enter same key again, the old key will be overwritten and only the new key will be available.
- Keys should be immutable type. For example, we can use a number, string or tuples as keys since they are immutable. We cannot use lists or dictionaries as keys. If they are used as keys, we will get „TypeError“.

Example:# Creating an empty Dictionary

```
Dict = {}

print("Empty Dictionary: ")

print(Dict)

# Creating a Dictionary

# with dict() method

Dict = dict({1: 'hello', 2: 'For', 3: 'hello'})

print("\nDictionary with the use of dict(): ")

print(Dict)

# Creating a Dictionary

# with each item as a Pair

Dict = dict([(1, 'hello'), (2, 'For')])

print("\nDictionary with each item as a pair: ")

print(Dict)
```

Output:
Empty Dictionary:
{}
Dictionary with the use of dict():
{1: 'hello', 2: 'For', 3: 'hello'}
Dictionary with each item as a pair:
{1: 'hello', 2: 'For'}

converting Lists into Dictionary:

When we have two lists, it is possible to convert them into a dictionary. For example, we have two lists containing names of countries and names of their capital cities.

There are two steps involved to convert the lists into a dictionary. The first step is to create a „zip“ class object by passing the two lists to zip() function. The zip() function is useful to convert the sequences into a zip class object. The second step is to convert the zip object into a dictionary by using dic() function.

Example:

```
def Convert(a):
    it = iter(a)
    res_dct = dict(zip(it,
it)) return res_dct
list = ['GERMANY', 'Berlin', 'INDIA', 'New Delhi', 'USA',
'Washington', 'FRANCE', 'Paris']
print(Convert(list))
```

Output:

```
{'GERMANY': 'Berlin', 'INDIA': 'New Delhi', 'USA': 'Washington', 'FRANCE': 'Paris'}
```

- **A Python program to create a dictionary and find the sum of values.**

```
dic={'A':100,'B':540,'C':239}
print("Total sum of values in the dictionary:")
print(sum(dic.values()))
output:
Total sum of values in the dictionary:
879
```

4.42.2. Dictionary Operations And Methods:**Dictionary Operations:****Accessing Elements from Dictionary**

While indexing is used with other data types to access values, a dictionary uses keys. Keys can be used either inside square brackets [] or with the get() method. If we use the square brackets [], KeyError is raised in case a key is not found in the dictionary. On the other hand, the get() method returns None if the key is not found.

```
Dic={1:"apple",2:"orange",3:"grapes"}
```

```
Print(Dic[2])
```

```
>>Orange
```

```
Print(Dic.get(1))
```

```
>>apple
```

Adds a new pair of key and value to the dictionary, but in case that the key already exists in the dictionary, we can update the value.

```
y = {}
```

```
y['one'] = 1
```

```
y['two'] = 2
```

```
print(y)
```

```
Output:{'one': 1, 'two': 2}
```

```
y['two'] = 'dos'
```

```
print(y)
```

```
Output: {'one': 1, 'two': 'dos'}
```

before, the key value should be an immutable data type, for that reason if you try to define a dictionary with a mutable data type, Python will raise a `TypeError: unhashable type exception`.

del

Del statement can be used to remove an entry (key-value pair) from a dictionary.

```
y = {'one': 1, 'two': 2}
print(y)
Output: {'one': 1, 'two': 2}
del y['two']
print(y)
Output: {'one': 1}
```

update

This method updates a first dictionary with all the key-value pairs of a second dictionary. Keys that are common to both dictionaries, the values from the second dictionary override those of the first.

```
x = {'one': 0, 'two': 2}
y = {'one': 1, 'three': 3}
x.update(y)
print(x)
Output: {'one': 1, 'two': 2, 'three': 3}
```

Dictionary Methods:

- **D.clear():** Removes all key-value pairs from dictionary D

Program	Output
D={"Regdno":556, "Name":"Mothi", "Branch":"CSE"} D.clear() print(D)	{}

- **D2=D1.copy():** Copies all elements from D1 into a new dictionary D2.

Program	Output
D1={"Regdno":556, "Name":"Mothi", "Branch":"CSE"} D2=D1.copy() print(D2)	{'Regdno': 556, 'Name': 'Mothi', 'Branch': 'CSE'}

- **D.keys():** Returns a sequence of keys from the dictionary D.

Program	Output
D={"Regdno":556, "Name":"Mothi", "Branch":"CSE"} print(D.keys())	dict_keys(['Regdno', 'Name', 'Branch'])

- **D.values():** Returns a sequence of values from the dictionary D.

Program	Output
D={"Regdno":556, "Name":"Mothi", "Branch":"CSE"} print(D.values())	dict_values([556, 'Mothi', 'CSE'])

- **D1.update(D2):** Add all elements from dictionary D2 to D1.

Program	Output
<pre>D1={"Regdno":556,"Name":"Mothi","Branch":"CSE"} D2={"Sub1":56,"Sub2":65,"Sub3":74} D1.update(D2) print(D1)</pre>	<pre>{'Regdno': 556, 'Name': 'Mothi', 'Branch': 'CSE', 'Sub1': 56, 'Sub2': 65, 'Sub3': 74}</pre>

- **D.pop(Key):** Removes the key and its value from D.

Program	Output
<pre>D={"Regdno":556,"Name":"Mothi","Branch":"CSE"} D.pop("Branch") print(D)</pre>	<pre>{'Regdno': 556, 'Name': 'Mothi'}</pre>

- **D.setdefault(K,V):** If key K is found, its value is returned. If key K is not found, then the K,V pair is stored into the dictionary D.

Program	Output
<pre>D={"Regdno":556,"Name":"Mothi","Branch":"CSE"} D.setdefault("Gender","Male") print(D)</pre>	<pre>{'Regdno': 556, 'Name': 'Mothi', 'Branch': 'CSE', 'Gender': 'Male'}</pre>

for loop is very convenient to retrieve the elements of a dictionary. Let's take a simple dictionary that contains color code and its name as:

```
colors = {'r':"RED", 'g':"GREEN", 'b':"BLUE", 'w':"WHITE" }
```

Here, „r“, „g“, „b“ represents keys and „RED“, „GREEN“, „BLUE“ and „WHITE“ indicate values.

4.12.2 dictionary comprehension:

Like List Comprehension, Python allows dictionary comprehensions. We can create dictionaries using simple expressions. A dictionary comprehension takes the form {key: value for (key, value) in iterable}

Example:

Here we have two lists named keys and value and we are iterating over them with the help of zip()

```
keys =
['a', 'b', 'c', 'd', 'e']
values = [1,2,3,4,5]
myDict = { k:v for (k,v) in zip(keys,
values)} print (myDict)
output:
{'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
```

Using fromkeys() Method

Here we are using the fromkeys() method that returns a dictionary with specific keys and values.

```
dic=dict.fromkeys(range(5),
True) print(dic)
```

Output:

```
{0: True, 1: True, 2: True, 3: True, 4: True}
```

Using dictionary comprehension make dictionary

Example 1:

```
# Python code to demonstrate dictionary
# creation using list comprehension
myDict = {x: x**2 for x in [1,2,3,4,5]}
print (myDict)
```

Output :

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Using conditional statements in dictionary comprehension

```
newdict = {x: x**3 for x in range(10) if x**3 % 4 == 0}
print(newdict)
```

Output :

```
{0: 0, 8: 512, 2: 8, 4: 64, 6: 216}
```

SET:

Set is another data structure supported by python. Basically, sets are same as lists but with a difference that **sets are with no duplicate entries**. Technically a set is a mutable and an unordered collection of items. This means that we can **easily** add or remove items from it.

Creating a Set:

A set is created by placing all the elements inside curly brackets {}. Separated by comma or by using the built-in function set().

Example:

```
set={1,3,4,5,1,6}
```

```
print(set)
```

```
output:{1,3,4,5,6}
```

Updating a set:

Since sets are unordered, indexing has no meaning. Set operations do not allow users to access or change an element using indexing or slicing.

Methods on set:

- **len(S)**: Returns the number of elements in set S.

Program	Output
A={1,2,3,4,5,6,7} print(len(A))	7

- **S1.issubset(S2)**: Test whether every element in S1 is in S2.

Program	Output
A={1,2,3,4} B={1,2,3,4,5,6,7} print(A.issubset(B))	True

- **S1.issuperset(S2)**: Test whether every element in S2 is in S1.

Program	Output
A={1,2,3,4} B={1,2,3,4,5,6,7} print(A.issuperset(B))	False

- **S1 == S2:** Returns True if both sets are equal.

Program	Output
A={1,2,3,4} B={1,2,3,4,5,6,7} print(A==B)	False

- **S1 != S2:** Returns True if both sets are not equal.

Program	Output
A={1,2,3,4} B={1,2,3,4,5,6,7} print(A!=B)	True

- **S1.union(S2):** Generate new sets with all elements from both S1 and S2.

Program	Output
A={1,2,3,4,5,6,7} B={4,5,6,7,8,9,10} C=A.union(B) print(C)	{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

- **S1.intersection(S2):** Generate new sets with common elements from both S1 and S2.

Program	Output
A={1,2,3,4,5,6,7} B={4,5,6,7,8,9,10} C=A.intersection(B) print(C)	{4, 5, 6, 7}

- **S1.difference(S2):** Generate new set with elements in S1 not in S2.

Program	Output
A={1,2,3,4,5,6,7} B={4,5,6,7,8,9,10} C=A.difference(B) print(C)	{1, 2, 3}

- **S1.symmetric_difference(S2):** Generate new set with elements in S1 not in S2 and elements in S2 not in S1.

Program	Output
A={1,2,3,4,5,6,7} B={4,5,6,7,8,9,10} C=A.symmetric_difference(B) print(C)	{1, 2, 3, 8, 9, 10}

- **S.copy():** Generate new shallow copy of set S.

Program	Output
A={1,2,3,4,5,6,7} B=A.copy() print(B)	{1, 2, 3, 4, 5, 6, 7}

- **S.add(X):** Add element X to set S.

Program	Output
---------	--------

A={1,2,3,4,5,6,7} A.add(10) print(A)	{1, 2, 3, 4, 5, 6, 7, 10}
--	---------------------------

- **S.discard(X):** Removes X from set S if X is present.

Program	Output
A={1,2,3,4,5,6,7} A.discard(5) print(A)	{1, 2, 3, 4, 6, 7}

- **S.remove(X):** Removes X from set S if X is present; raises KeyError if not present.

Program	Output
A={1,2,3,4,5,6,7} A.remove(5) print(A)	{1, 2, 3, 4, 6, 7}

- **S.pop():** Removes and arbitrary element (Randomly element) from S.

Program	Output
A={10,15,12,21,3,4,5,6,7} A.pop() print(A)	{4, 5, 6, 7, 10, 12, 15, 21}

- **S.clear():** Removes all elements from set S.

Program	Output
A={10,15,12,21,3,4,5,6,7} A.clear() print(A)	set()

- **max(S):** Returns maximum element from S.

Program	Output
A={10,15,12,21,4,5,6,7} print(max(A))	21

- **min(S):** Returns minimum element from S.

Program	Output
A={10,15,12,21,4,5,6,7} print(min(A))	4

- **sorted(S):** Returns a new sorted list from the elements in the set S.

Program	Output
A={10,15,12,21,4,5,6,7} B=sorted(A) print(B)	[4, 5, 6, 7, 10, 12, 15, 21]